

TP : Déploiement d'un Datacenter Conteneurisé Sécurisé

Public : Étudiants en BTS SISR (Services Informatiques aux Organisations)

Objectifs : Maîtriser l'Infrastructure as Code (IaC), le Reverse Proxy, la segmentation réseau Docker et la supervision proactive.

Mise en situation professionnelle

Vous êtes Administrateur Système Junior chez *LycéeCorp*. Le DSI vous demande de moderniser l'infrastructure en abandonnant les installations manuelles. Vous devez déployer un gestionnaire de parc (GLPI) avec sa base de données, mais avec des contraintes strictes : la base de données doit être invisible depuis Internet, le site doit être accessible via un nom de domaine, et l'infrastructure doit s'auto-réparer en cas de panne matérielle.

Phase 1 : Préparation du terrain (Clean Slate)

En administration système, un port réseau (comme le port 80 pour le Web) ne peut être écouté que par un seul programme à la fois. Si un ancien test tourne encore, notre nouveau proxy plantera au démarrage.

1. **Listez tous les conteneurs (actifs et inactifs) :**

```
docker ps -a
```

2. **Nettoyez les anciens conteneurs bloquants :** Repérez tout conteneur (ex: nginx, hello-world) et forcez sa suppression :

```
docker rm -f <nom_du_conteneur>
```

3. **Créez un espace de travail propre :** On ne travaille jamais à la racine du système Linux.

```
4. mkdir ~/datacenter  
cd ~/datacenter
```

Phase 2 : L'Architecture as Code (Docker Compose)

Théorie : Le réseau virtuel Docker

Par défaut, un conteneur est aveugle. Nous allons créer deux réseaux :

- `reseau_public` : Le réseau ouvert où le Proxy accueille les utilisateurs.
- `reseau_privé` : Un VLAN logiciel strictement isolé. La ligne `internal: true` garantit qu'aucun paquet réseau ne peut y entrer ni en sortir depuis l'extérieur. Idéal pour une base de données !

Créez le fichier de configuration : `nano docker-compose.yml` et collez l'architecture complète ci-dessous :

```
#
=====
# 🏢 INFRASTRUCTURE DATACENTER - FICHIER DOCKER COMPOSE
#
=====
services:

# 🔄 1. LE REVERSE PROXY : NGINX PROXY MANAGER (NPM)
# Rôle : Le standardiste. C'est le SEUL conteneur ouvert vers
l'extérieur.
npm:
  image: 'jc21/nginx-proxy-manager:latest'
  container_name: proxy-npm
  restart: unless-stopped # Redémarrage auto en cas de panne du serveur
physique
  ports:
    - '80:80' # Écoute les requêtes HTTP d'Internet
    - '443:443' # Écoute les requêtes HTTPS sécurisées
    - '81:81' # Port d'accès à l'interface d'administration
  networks:
    - reseau_public
  volumes:
    - npm_data:/data
    - npm_letsencrypt:/etc/letsencrypt

# 🗄️ 2. LA BASE DE DONNÉES : MARIADB
# Rôle : Le coffre-fort. Stocke toutes les données (tickets, mots de
passe).
db_glpi:
  image: mariadb:10.6
  container_name: glpi-db-secure
  restart: unless-stopped
```

```

environment:
  MYSQL_ROOT_PASSWORD: root_password_super_secret
  MYSQL_DATABASE: glpidb
  MYSQL_USER: glpi_user
  MYSQL_PASSWORD: glpi_password
networks:
  - reseau_prive          # ⚠ SÉCURITÉ : Totalement coupée d'Internet
volumes:
  - db_data:/var/lib/mysql

# 📦 3. L'APPLICATION MÉTIER : GLPI
# Rôle : L'interface web. Fait le pont entre le Proxy et la Base de
Données.
app_glpi:
  image: diouxx/glpi:latest
  container_name: glpi-app-web
  restart: unless-stopped
  networks:
    - reseau_public        # Pour recevoir les clients depuis le Proxy
    - reseau_prive         # Pour pouvoir lire dans la Base de Données
  volumes:
    - glpi_data:/var/www/html/glpi
  depends_on:
    - db_glpi              # Attendre que la DB soit prête avant de
s'allumer

# ☹ 4. LA SUPERVISION : UPTIME KUMA
# Rôle : Surveiller l'état de nos services en temps réel.
monitoring:
  image: 'louislam/uptime-kuma:1'
  container_name: monitoring-kuma
  restart: unless-stopped
  ports:
    - '3001:3001'
  networks:
    - reseau_public
  volumes:
    - kuma_data:/app/data

#
=====
=====
# 🌐 RÉSEAUX VIRTUELS ET 📁 VOLUMES DE PERSISTANCE
#
=====
=====
networks:
  reseau_public:
    name: reseau_public
  reseau_prive:
    name: reseau_prive
    internal: true          # LIGNE MAGIQUE : Coupe totalement ce réseau
d'Internet

```

```
volumes:
  npm_data:
  npm_letsencrypt:
  db_data:
  glpi_data:
  kuma_data:
```

Phase 3 : Le Déploiement

Ordonnez à Docker de lire la partition et de lancer l'orchestre en arrière-plan (mode Detached) :

```
docker compose up -d
```

Vérifiez que vos 4 conteneurs sont au statut **Up** :

```
docker compose ps
```

```
mondher@debian13-server:~/datacenter$ docker compose ps
```

NAME	PORTS	IMAGE	COMMAND	SERVICE	CREATED	STATUS
glpi-app-web	80/tcp, 443/tcp	diouxx/glpi:latest	"/opt/glpi-start.sh"	app_glpi	4 minutes ago	Up 4 minutes
glpi-db-secure	3306/tcp	mariadb:10.6	"docker-entrypoint.s..."	db_glpi	4 minutes ago	Up 4 minutes
monitoring-kuma		louislam/uptime-kuma:1	"/usr/bin/dumb-init ..."	monitoring	4 minutes ago	Up 4 minutes (he
proxy-npm	0.0.0.0:80-81->80-81/tcp, [::]:80-81->80-81/tcp, 0.0.0.0:443->443/tcp, [::]:443->443/tcp	jc21/nginx-proxy-manager:latest	"/init"	npm	4 minutes ago	Up 4 minutes

```
mondher@debian13-server:~/datacenter$
```

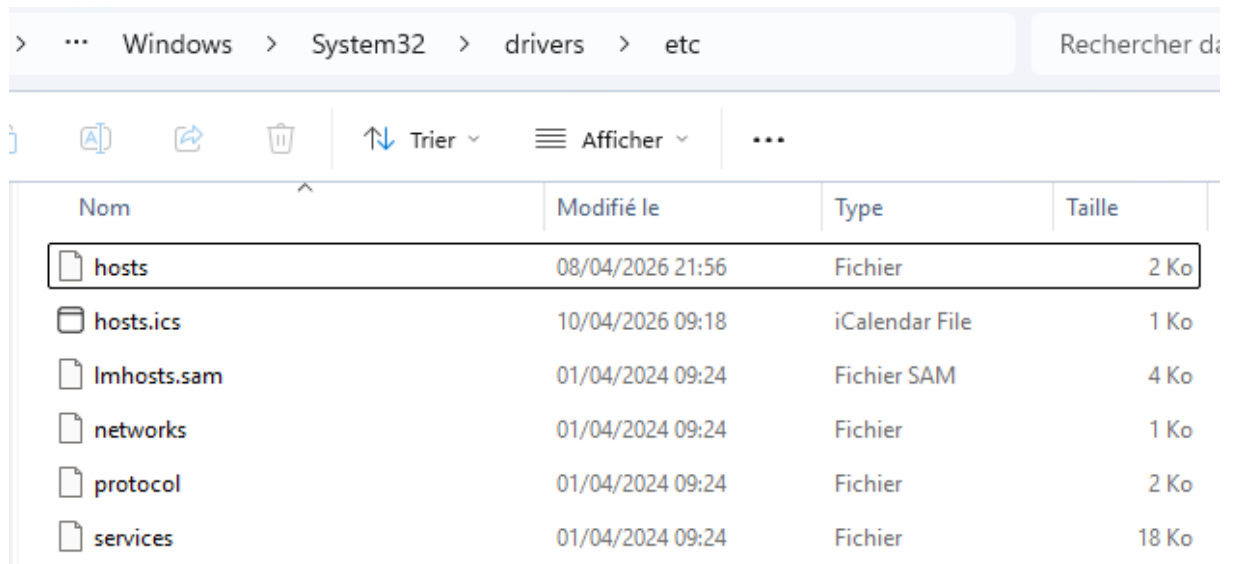
Phase 4 : Routage DNS et Configuration du Proxy

L'objectif d'un Datacenter professionnel est de ne jamais obliger un utilisateur à taper une adresse IP. Tout doit passer par un nom de domaine propre.

Étape 4.1 : Le DNS Local (Sur votre PC physique)

Pour simuler l'achat d'un nom de domaine, nous allons tromper votre ordinateur Windows/Mac :

1. Ouvrez le logiciel **Bloc-notes en tant qu'Administrateur**.
2. Ouvrez le fichier système : `C:\Windows\System32\drivers\etc\hosts`
3. Ajoutez cette ligne tout en bas (remplacez par l'IP de votre serveur Debian) :
`192.168.x.x glpi.local`
4. Enregistrez le fichier.



hosts

×

+

Fichier

Modifier

Affichage

H1

⋮

B

```

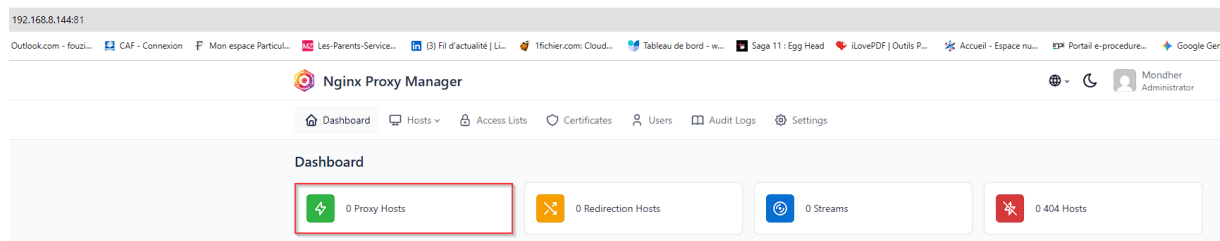
# Copyright (c) 1993-2009 Microsoft Corp.
#
# This is a sample HOSTS file used by Microsoft TCP/IP for Windows.
#
# This file contains the mappings of IP addresses to host names. Each
# entry should be kept on an individual line. The IP address should
# be placed in the first column followed by the corresponding host name.
# The IP address and the host name should be separated by at least one
# space.
#
# Additionally, comments (such as these) may be inserted on individual
# lines or following the machine name denoted by a '#' symbol.
#
# For example:
#
#       102.54.94.97       rhino.acme.com       # source server
#       38.25.63.10       x.acme.com          # x client host
#
# localhost name resolution is handled within DNS itself.
#       127.0.0.1         localhost
#       ::1               localhost
# Added by Docker Desktop
192.168.1.181 host.docker.internal
192.168.1.181 gateway.docker.internal
# To allow the same kube context to work on the host and the container:
127.0.0.1 kubernetes.docker.internal
# End of section
192.168.8.144 glpi.local

```

Étape 4.2 : Paramétrage du Reverse Proxy

Maintenant, il faut expliquer au gardien (NPM) à qui donner le trafic quand quelqu'un demande *glpi.local*.

1. Sur votre PC, allez sur : http://<IP_DE_VOTRE_DEBIAN>:81
2. Connectez-vous avec **admin@example.com** (et changez les identifiants).



3. Allez dans **Hosts** > **Proxy Hosts** > **Add Proxy Host**.
4. Remplissez strictement ainsi :
 - **Domain Names** : [glpi.local](#) (Appuyez sur Entrée pour valider la bulle)
 - **Scheme** : [http](#)
 - **Forward Hostname / IP** : [glpi-app-web](#) (La magie de Docker : On utilise le nom du conteneur en guise d'IP !)
 - **Forward Port** : [80](#)
 - Cochez : *Cache Assets*, *Block Common Exploits* et *Websockets Support*.

The image shows the 'Add Proxy Host' form in Nginx Proxy Manager. It has tabs for 'Details', 'Custom Locations', and 'SSL'. The 'Details' tab is active. The form contains the following fields:

- Domain Names**: A text input with 'glpi.local' and a dropdown arrow. This field is highlighted with a red box.
- Scheme**: A dropdown menu with 'http' selected. This field is highlighted with a red box.
- Forward Hostname / IP**: A text input with 'glpi-app-web'. This field is highlighted with a red box.
- Forward Port**: A text input with '80'. This field is highlighted with a red box.
- Access List**: A dropdown menu with 'Publicly Accessible' selected.
- Options**: Three toggle switches: 'Cache Assets', 'Block Common Exploits', and 'Websockets Support'. All three are turned on. This section is highlighted with a red box.

At the bottom, there are 'Cancel' and 'Save' buttons.

5. Cliquez sur **Save**.

Phase 5 : L'Installation de GLPI et le piège du réseau

⚠ Attention au serveur SQL

Ouvrez un nouvel onglet sur **http://glpi.local**. Lors de l'installation, à l'étape de la base de données, l'erreur du débutant est de taper **localhost**. Or, la base de données n'est pas dans le même conteneur !

Remplissez les champs de connexion à la base ainsi :

- **Serveur SQL** : **glpi-db-secure** (Le DNS interne de Docker traduira ce nom)
- **Utilisateur** : **glpi_user**
- **Mot de passe** : **glpi_password**

Phase 6 : Sécurisation Post-Installation

Une fois sur le tableau de bord GLPI, une alerte orange vous demande de supprimer le fichier **install.php**. Pour intervenir à l'intérieur d'un conteneur en cours d'exécution, on utilise la commande **exec** depuis le serveur Debian :

```
docker exec -it glpi-app-web rm /var/www/html/glpi/install/install.php
```

Rafraîchissez la page GLPI, l'alerte a disparu.

Phase 7 : La Supervision Proactive (Uptime Kuma)

En entreprise, la disponibilité des services est souvent encadrée par des contrats (SLA). Un SysAdmin ne doit jamais apprendre par ses utilisateurs qu'un service est en panne. Kuma agit comme un robot qui interroge nos applications toutes les 20 secondes.

1. Connectez-vous à la supervision : **http://<IP_DE_VOTRE_DEBIAN>:3001**
2. Créez votre compte Administrateur Kuma.
3. Cliquez sur + **Ajouter un moniteur** :
 - **Type** : HTTP(s)
 - **Nom** : GLPI Interne
 - **URL** : **http://glpi-app-web** (Kuma étant sur le même réseau privé *public_network*, il peut pinger GLPI directement sans passer par Internet !)

- **Intervalle : 20 secondes**

4. **Le métier de SysAdmin :** Allez dans l'onglet *Notifications* de Kuma. Observez que vous pouvez lier ce moniteur à un serveur Discord, Teams ou Slack. En cas de panne, vous recevrez un message sur votre téléphone avant même que les utilisateurs ne s'en rendent compte !

Uptime Kuma Mise à jour disponible Pages de statut Tableau de bord

Ajouter une nouvelle sonde

Général

Type de sonde: HTTP(s)

Nom d'affichage: GLPI interne

URL: http://glpi-app-web

Intervalle de vérification (Vérifier toutes les 20 secondes): 20

Essais: 0

Notifications

Non disponible, merci de le configurer.

Créer une notification

Proxy

Non disponible, merci de le configurer.

Configurer le proxy

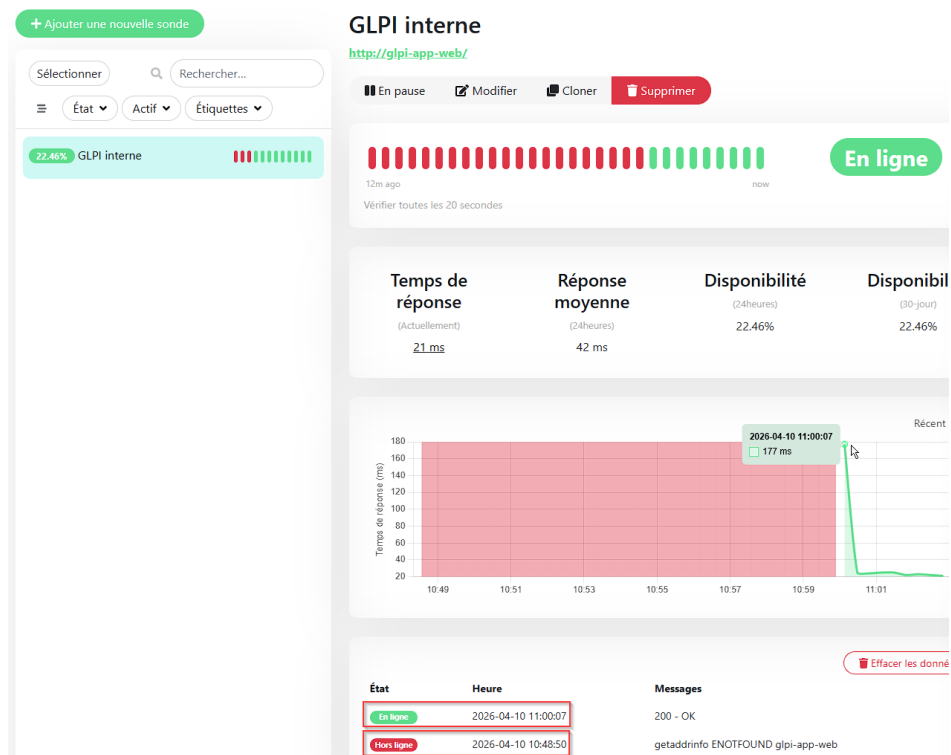
Options HTTP

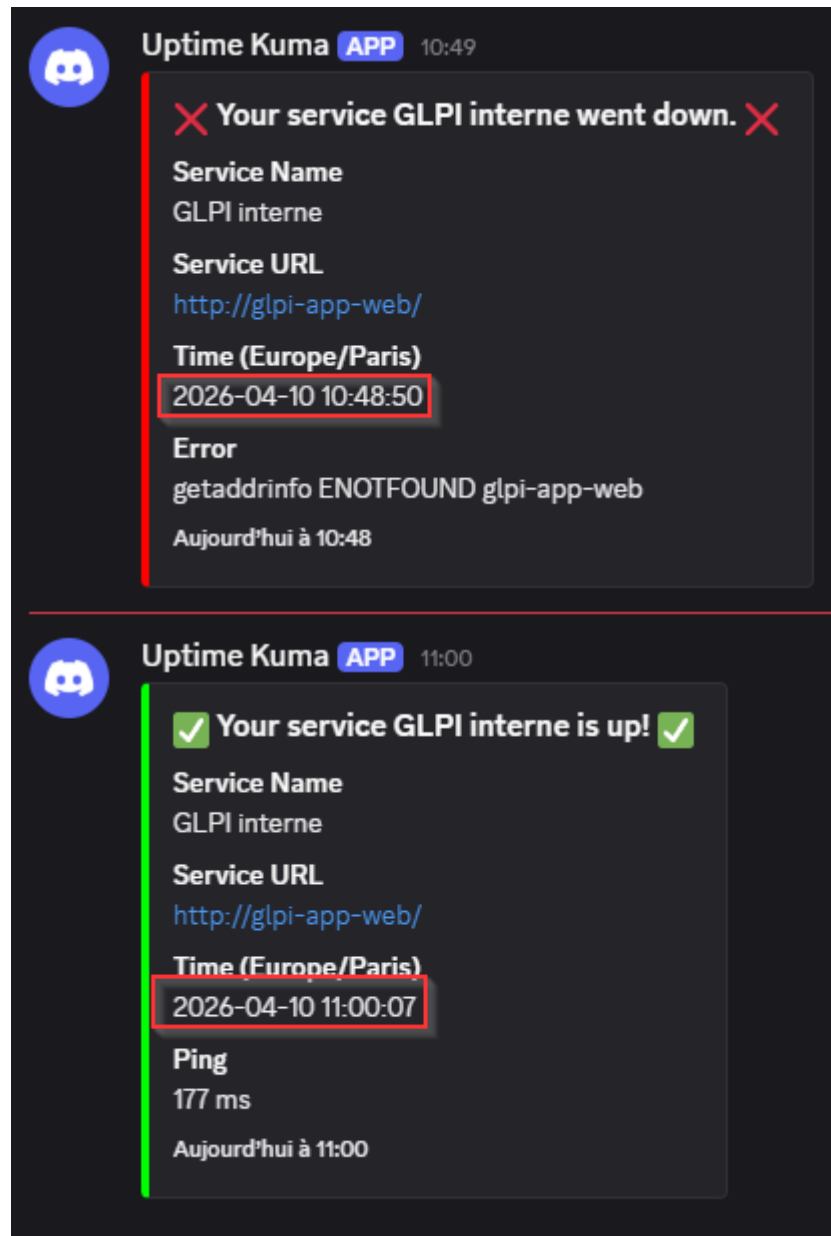
Méthode: GET

5. Faisons un test : stoppez le conteneur GLPI avec la commande

```
docker stop glpi-app-web
```

Uptime Kuma l'a immédiatement détecté et a envoyé une alerte sur Discord !





Phase 8 : Le Crash Test (Auto-guérison)

L'épreuve ultime. Si votre baie de serveur perd le courant, votre infrastructure doit pouvoir remonter seule dès le retour de l'électricité.

1. Depuis votre terminal Debian, forcez un redémarrage brutal :

```
sudo reboot
```

2. Attendez 1 minute que le serveur physique redémarre.
3. Reconnectez-vous et tapez `docker ps`.

✓ Mission Accomplie

Grâce aux directives `restart: unless-stopped`, Docker a relancé tous les services.
Grâce aux `volumes` configurés en fin de fichier, aucune donnée n'a été perdue. Votre Datacenter est résilient !